

Open-MOPD: Diagnosing and Fixing Capability Imbalance in Multi-Teacher On-Policy Distillation

Huan-ang Gao^{*,†,1,2,3}, Haohan Chi^{*,1,2,3}, Yong Yan^{1,2,3}, Shiyuan Feng^{1,2},
Hanlin Wu^{1,2}, Zheng Jiang³, Bingxiang He³, Wei-Ying Ma^{1,2}, Ya-Qin
Zhang^{1,2}, Hao Zhou^{1,2,†}

¹SIA-Lab of Tsinghua AIR and ByteDance Seed

²Institute for AI Industry Research (AIR), Tsinghua University

³Department of Computer Science and Technology, Tsinghua University

*Equal contribution, [†]Project Lead, [†]Corresponding author

Abstract

Multi-teacher on-policy distillation (M-OPD) has emerged as a promising paradigm for consolidating domain-specialized reinforcement learning (RL) experts into a single generalist student via dense, token-level reward supervision. Despite its practical success, the optimization dynamics governing multi-teacher capability integration remain poorly understood, and open, rigorously reproducible recipes are conspicuously lacking. In this work, we establish a controlled M-OPD benchmark on SmoLM3-3B-Base with oracle routing, isolating capability integration from routing ambiguity. Our investigation reveals a pronounced *capability integration gap*: standard M-OPD recovers only 35.6% of the improvement from mixed-domain SFT to RouteRL. Concise tasks such as instruction following suffer severe degradation and premature stagnation. Under this metric, token-level teacher disagreement does not appear to be the dominant bottleneck; the main failure is a severe misallocation of the token-level optimization budget. We identify three separable contributors to this imbalance: structural sequence-length disparities across domains, different convergence rates across domains, and reward staleness from stale student updates across repeated minibatches of a shared rollout. To resolve these imbalances, we introduce **Open-MOPD**, a principled framework incorporating token-share balancing, gap-aware dynamic budget allocation, and student reward refresh. Together, these mechanisms systematically restore cross-domain balance, elevating recovery from 35.6% to 83.4% of the improvement from mixed-domain SFT to RouteRL in one student model. We fully open-source our end-to-end post-training recipe, training trajectories, and evaluation suites on an 8×A100-80GB academic setup.

Project page: <https://bytedtsinghua-sia.github.io/Open-MOPD/>

1 Introduction

Reinforcement learning (RL) has established itself as a foundational pillar in large language model (LLM) post-training, excelling at cultivating specialized capabilities in domain-specific models [1, 2]. However, serving multiple specialized models in real-world applications is often computationally prohibitive, creating a strong demand for a single unified model that preserves these disparate strengths. To consolidate multiple specialists into a single student, multi-teacher on-policy distillation (M-OPD) has emerged as an appealing paradigm; each input prompt is routed to a corresponding domain expert, which provides dense per-token reward supervision over the trajectories generated by the student policy [3–5]. Despite the increasing adoption of multi-teacher distillation in industrial pipelines, the fundamental mechanisms governing multi-domain capability integration

remain poorly understood, and the research community still lacks an open and rigorously reproducible recipe.

To isolate and examine the core dynamics of multi-teacher consolidation, we construct a fully controlled M-OPD experimental framework initialized from SmolLM3-3B-Base. The pipeline incorporates a three-domain mixed-SFT initialization, three domain-expert RL teachers (spanning mathematics, coding, and general instruction following), and oracle routing based on ground-truth domain labels. Using ground-truth domain labels, we isolate capability integration from routing errors. This design operates at a parameter scale that balances two criteria (Section 2.2), and the entire pipeline and ablation ladder are reproducible on an 8×A100-80GB academic setup, while the model remains capable of long-horizon reasoning.

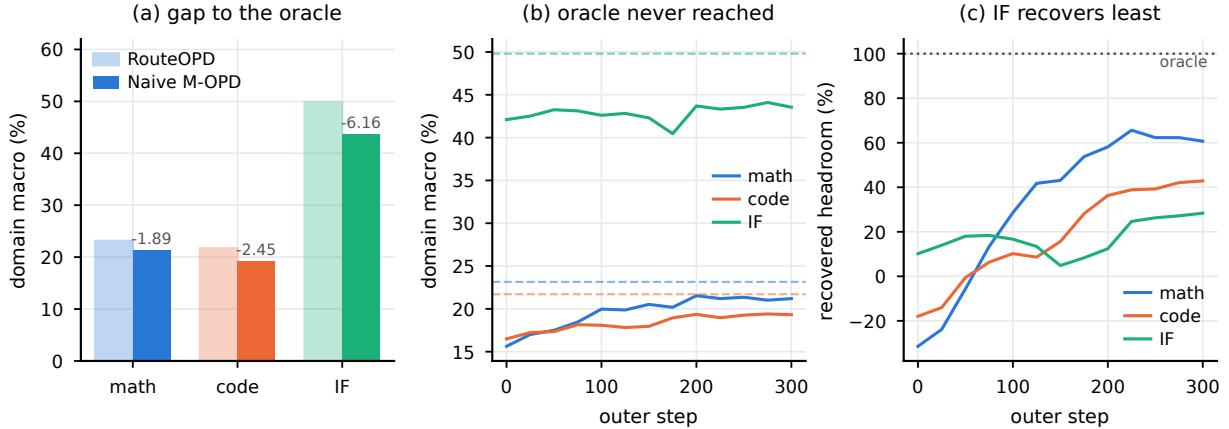


Figure 1 The integration gap is real and strongly asymmetric. (a) Per-domain macro-averages of Naive M-OPD and RouteOPD, where the number above each bar indicates the gap in that domain; (b) per-domain validation trajectories of Naive M-OPD, with dashed lines denoting the RouteOPD reference for the corresponding domain; (c) recovered theoretical headroom relative to RouteRL, $(\text{current} - \pi_{\text{mixsft}}) / (\text{RouteRL} - \pi_{\text{mixsft}})$. Instruction following (IF) exhibits the largest absolute gap and the lowest recovered headroom, and is the earliest domain to stop improving.

Our empirical investigation reveals that perfect routing is far from sufficient for successful capability integration. Under a standardized six-benchmark evaluation protocol, naive M-OPD reaches an average score of 28.05, while distilling each domain on its own reaches 31.55. We use RouteOPD to measure the deployment-time integration gap. In particular, concise instruction-following (IF) tasks fall 6.16 points below their RouteOPD reference ($3.3\times$ the degradation observed in mathematics) and plateau earliest during training.

Through systematic per-domain measurement, we locate this integration gap in the unmanaged allocation of the token-level optimization budget rather than catastrophic gradient conflict (Section 3). Because on-policy distillation objectives aggregate loss values across tokens, the actual optimization share received by each domain is governed by its gradient token volume rather than its prompt frequency. Consequently, concise IF responses contribute a negligible fraction of gradient tokens despite receiving a balanced prompt allocation, accounting for 20.3% of the input prompts but only 0.99% of the gradient tokens. Furthermore, even if the raw token budget is equalized initially, it drifts apart within tens of steps because the supervisory reward signals diminish at uneven rates as the student converges toward different teachers at varying speeds. This imbalance is compounded by standard multi-step rollout reuse, which leaves student-dependent reward components stale across successive gradient updates.

To address these orthogonal failure modes, we introduce Open-MOPD, a principled framework that targets each distortion with a dedicated mechanism (Section 4). First, *token-share balancing* decouples gradient budget allocation from sequence length, establishing a weighted token-share target rather than a fixed per-batch quota. Second, *gap-aware allocation* dynamically steers the optimization budget toward domains with the largest remaining student-teacher gap. This prevents the training collapse observed with naive reward normalization at step 74, where budget is wastefully funneled into already-converged domains. Third, *student reward refresh* recomputes student log-probabilities before each gradient step while caching teacher states, eliminating sample

staleness with negligible overhead [6]. In cumulative ablation experiments, Open-MOPD raises recovery from 35.6% to 83.4% in one student model.

Our principal contributions are summarized below.

- **Empirical Diagnosis.** Using an oracle-routed testbed that isolates integration dynamics from routing errors, we identify and characterize the *capability integration gap* in multi-teacher on-policy distillation, revealing pronounced cross-domain performance asymmetry and premature stagnation in concise tasks (Section 3).
- **Mechanistic Decomposition.** Token-level teacher disagreement under our metric is unlikely to be the dominant bottleneck. We instead identify three separable contributors to token-level optimization budget distortion: sequence-length disparities, different convergence rates, and reward staleness from stale student updates across repeated minibatches of a shared rollout (Section 4).
- **Principled Methodology.** We propose Open-MOPD, an optimization framework combining token-share balancing, gap-aware dynamic budget allocation, and student reward refresh, which resolves domain imbalances and recovers most of the available headroom in one student model (Section 4, Section 5).
- **Open-Source Recipe.** We fully open-source our end-to-end post-training pipeline, model checkpoints, and evaluation suites on a reproducible academic compute budget (Appendix A).

2 Open-MOPD: An Open Recipe for Multi-Teacher Capability Integration

Our goal is to integrate the capabilities of several domain experts into one unified student model. Let the set of domains be $\mathcal{D}$. On our base model, $\mathcal{D} = \{\text{math}, \text{code}, \text{instruction_following (IF)}\}$. Each domain has an expert teacher π_{ϕ_d} trained with reinforcement learning, and the student model is π_θ . Each training sample carries a domain label $d(x)$, allowing a single student model to receive supervision signals from the corresponding domain expert across three distinct tasks: mathematics, code generation, and instruction following. With the training framework fixed, Open-MOPD aims to identify a recipe that stably transfers the capabilities of all three experts into a single student model.

2.1 The Multi-Teacher On-Policy Distillation Objective

For a prompt x , the student first samples a response $y \sim \pi_\theta(\cdot | x)$ from the current policy. The teacher evaluates student rollouts via a single prefill pass to generate the teacher distribution. Write the routed teacher as $\pi_{\phi_{d(x)}}$. At position t , we take the top- k token set of the student distribution

$$\mathcal{S}_t = \text{TopK}_k(\pi_\theta(\cdot | x, y_{<t})), \quad (1)$$

with $k = 16$ by default. For each $v \in \mathcal{S}_t$, we first define the teacher-student log-probability difference

$$\delta_t(v) = \text{sg}\left[\log \pi_{\phi_{d(x)}}(v | x, y_{<t}) - \log \pi_\theta(v | x, y_{<t})\right], \quad (2)$$

where $\text{sg}[\cdot]$ denotes stop-gradient; this is the token-level advantage signal of dense distillation. It is then aggregated with softmax weights over $\mathcal{S}_t$ into a dense reward

$$r_t(v) = \delta_t(v) \cdot \tilde{\pi}_\theta(v | x, y_{<t}), \quad \tilde{\pi}_\theta(v | x, y_{<t}) = \text{softmax}_{u \in \mathcal{S}_t}[\log \pi_\theta(u | x, y_{<t})](v), \quad (3)$$

and tokens outside $\mathcal{S}_t$ contribute zero. The position-level reward is

$$r_t = \sum_{v \in \mathcal{S}_t} r_t(v). \quad (4)$$

With no critic, r_t is placed directly in the advantage slot of PPO. Intuitively, $\delta_t(v) > 0$ means that the teacher assigns a higher probability to token v than the student does; $\tilde{\pi}_\theta$ only determines the share each token inside the top- k contributes to the position-level reward r_t . Because the student generates every response, the

supervision distribution follows the student’s current policy. This is the key difference between on-policy distillation and supervised fine-tuning on offline teacher trajectories, whose fixed data distribution can create a distribution shift. For this reason, Section 3 starts from this minimal version, which adds no further training mechanisms and keeps the effects of later changes easier to identify.

2.2 Base Model Selection

The open recipe uses SmolLM3-3B-Base as its base model. It is a fully open 3B decoder-only model whose pre-training covers web, math and code data and which is trained at a 64K context [7]. This choice balances compute cost with the feasibility of running the full recipe and repeating its ablations.

Experimental feasibility. Multi-teacher OPD needs student generation, one or more teacher forwards, long-response training and per-domain validation at once; a full reproduction also includes mixed-domain SFT, three domain RL teachers and the final multi-teacher distillation. If the base model started at 7B or larger, that chain would be hard to close within a budget on which an ordinary academic team can ablate repeatedly: the GPU hours taken by a single experiment rise, and the repeated runs needed for mechanism comparisons become unaffordable. Open-MOPD therefore takes “runnable on a single 8×A100-80GB node” as a recipe constraint, so that the end-to-end pipeline and the mechanism ablations can be executed repeatedly with limited computational resources.

Model capacity. The base model must also have enough capacity. A model that is too small lets the response length limit truncate long-chain supervision before it enters a usable trajectory; for a failed experiment, it is difficult to determine whether the failure is caused by the algorithm or training configuration, or by the model’s limited ability to discover and learn solutions within the response length limit. We observed this failure mode earlier with the smaller Qwen3-1.7B-Base as base model: across five epochs of SFT on the same OpenR1-Math-93k, its best AIME24 result was only 7.08%, and even with a 31K generation budget the truncation rate stayed at 69.17–80.42%. Qwen2.5-7B-Base on the same data reached 31.25%, with the truncation rate down to 12.92% (Appendix A.1). When most responses are truncated, a failed run reveals little about whether the training method itself is effective. SmolLM3-3B-Base is large enough to learn from the long responses used in our math and code SFT, which has a 32,768-token limit. This lets us measure the integration gap and run the ablations without response truncation dominating the results.

2.3 The End-to-End Training Recipe

The Open-MOPD recipe has three stages. The teachers in Stage II and the student in Stage III are initialized from the same mixed-domain SFT model. Table 1 summarizes the base model, the data, and the evaluation settings of the recipe.

Stage I: mixed-domain SFT. Starting from SmolLM3-3B-Base, we balance the sample count of each domain by response token count and then run four epochs of supervised fine-tuning on the mixed data, giving the shared student checkpoint π_{mixsft} . The data mixture is: math OpenR1-Math-93k (93,733 examples), code OCR-50k sampled from the full OpenCodeReasoning set (50,000 examples), and instruction-following Instruction-Nemotron aligned (820,039 examples); estimated by response tokens, the shares are approximately 37% math, 28% code and 35% IF. The sequence length limit is 32,768.

Stage II: per-domain RL teachers. Three teachers are initialized separately from π_{mixsft} , each running RL only on the verifiable reward of its own domain. The math teacher is trained on DAPO-Math-17k; the code teacher is trained on DeepScaler-24k (the LiveCodeBench-decontaminated version); the IF teacher is trained on Nemotron-IF-RL-46k. This stage gives $\pi_{\phi_{\text{math}}}, \pi_{\phi_{\text{code}}}, \pi_{\phi_{\text{IF}}}$.

Stage III: multi-teacher on-policy distillation. The student is also initialized from π_{mixsft} . Training prompts come from the union of the three domains and are sampled as a domain mixture. The student π_{θ} generates a response for each sampled prompt. The domain label then selects the corresponding teacher for OPD using

Table 1 The Open-MOPD pipeline. Starting from a mixed-domain SFT checkpoint (π_{mixsft}), domain-specific RL produces three teacher models π_{ϕ_d} . Multi-teacher OPD then trains the student model π_{θ} from the initial checkpoint. All datasets are publicly released for end-to-end reproducibility.

Stage	Data	Key settings and products
Base model	SmolLM3-3B-Base	3B; 1.7B fails to close long trajectories, while 7B is too costly for repeated ablations (Appendix A.1)
I Mixed-domain SFT	Math: OpenR1-Math-93k Code: OCR-50k (sampled from the full set) IF: Instruction-Nemotron	Four epochs; sequence limit 32,768; approximately balanced by response tokens (37/28/35) product π_{mixsft}
II Domain RL teachers	Math: DAPO-Math-17k Code: DeepScaler-24k (LCB-decontaminated) IF: Nemotron-IF-RL-46k	Independent RL per domain, no domain mixing; all forked from π_{mixsft} products $\pi_{\phi_{\text{math}}}, \pi_{\phi_{\text{code}}}, \pi_{\phi_{\text{IF}}}$
III Multi-teacher OPD	The union of the three domains’ RL prompts above; each prompt is hard-routed to its teacher by domain label	Response limit 16K for math/code and 2K for IF; top- $k=16$; the student is initialized from π_{mixsft} product π_{θ}
Evaluation	Math: AIME24/ AIME25 Code: LiveCodeBench v5/v6 IF: IFEval+ IFBench_{test}	mean@64 / mean@10 / mean@1; averaged evenly across the datasets within a domain first, then macro-averaged over the three domains

the objective in Section 2.1. The response length limit is 16K for math and code and 2K for IF; by default the dense reward is computed on the student top- k ($k = 16$).

2.4 Evaluation Setup

For each domain, we use the best performance obtained by training a student with its domain teacher alone as a reference. These three single-teacher students together form RouteOPD, our deployment-time integration-gap reference. At evaluation time, the domain label selects the corresponding student to generate the response. The performance gap between the unified multi-teacher OPD (M-OPD) student and RouteOPD defines the integration gap.

All main results use the same online verifier and the same six benchmarks. Scoring is done per dataset first, then averaged simply over the datasets within a domain, and the final total score is the simple average of the three domain averages. The math domain uses AIME24 and AIME25 with $n = 64$ and temperature = 0.6, reporting accuracy mean@64; the instruction-following domain uses IFEval and IFBench_{test} (abbreviated IFB_{test} in tables) with $n = 1$, reporting accuracy mean@1; the code domain uses LiveCodeBench v5 and v6 with $n = 10$ and temperature = 1.0, reporting accuracy mean@10.

Table 2 provides the reference results for SmolLM3-3B under this evaluation protocol. It includes the base model, the four mixed-domain SFT checkpoints, the single-domain RL and OPD results, and their domain-routed combinations RouteRL and RouteOPD. It also includes the RFT, π_{mixrl} , Parameter Merging, Naive M-OPD, and Open-MOPD comparisons. The analysis and ablations below use this table as their common reference.

Table 2 Integration gap on SmoLM3-3B. Each domain reports its sub-dataset scores and macro-average. We define the total score as the average of the three domain averages. The four SFT rows are checkpoints after successive epochs. Single-domain RL and OPD report results only for their respective domains. Gray-shaded *RouteRL* and *RouteOPD* use a separate model for each domain and select that model using the domain label at evaluation time. They therefore cannot be deployed as one model. The recovery rate is $\frac{\text{Row Total}-\text{SFT}}{\text{RouteRL}-\text{SFT}}$, using π_{mixsft} (epoch 4) as the SFT baseline. In the *Baselines* and *M-OPD* sections, the highest score in each column is boldfaced and the second highest is underlined.

Method	Math (avg@64)			Code (avg@10)			IF			Total	Recovery rate
	AIME24	AIME25	avg	LCBv5	LCBv6	avg	IFEval	IFB _{test}	avg		
SmoLM3-3B-Base	2.08	1.72	1.90	3.41	6.17	4.79	16.08	13.00	14.54	7.08	—
<i>SFT</i>											
π_{mixsft} (e1)	12.24	16.61	14.43	13.29	17.20	15.25	64.70	16.00	40.35	23.34	—
π_{mixsft} (e2)	12.55	17.97	15.26	13.17	16.63	14.90	65.62	16.00	40.81	23.66	—
π_{mixsft} (e3)	15.10	19.53	17.32	14.97	18.57	16.77	66.36	18.00	42.18	25.42	—
π_{mixsft} (e4)	15.63	20.26	17.95	15.99	19.20	17.60	66.91	16.00	41.46	25.67	—
<i>RL</i>											
$\pi_{\phi_{\text{math}}}$	23.65	24.84	24.24	—	—	—	—	—	—	—	—
$\pi_{\phi_{\text{code}}}$	—	—	—	22.16	21.31	21.73	—	—	—	—	—
$\pi_{\phi_{\text{IF}}}$	—	—	—	—	—	—	74.49	27.67	51.08	—	—
RouteRL	23.65	24.84	24.24	22.16	21.31	21.73	74.49	27.67	51.08	32.35	100%
<i>OPD</i>											
Math-OPD	22.34	23.96	23.15	—	—	—	—	—	—	—	—
Code-OPD	—	—	—	22.28	21.14	21.71	—	—	—	—	—
IF-OPD	—	—	—	—	—	—	75.60	24.00	49.80	—	—
RouteOPD	22.34	23.96	23.15	22.28	21.14	21.71	75.60	24.00	49.80	31.55	88.0%
<i>Baselines</i>											
RFT	22.97	23.91	23.44	18.98	19.43	19.21	55.08	18.67	36.87	26.51	12.6%
π_{mixrl}	21.15	22.14	21.64	16.59	20.97	18.78	70.24	<u>22.67</u>	46.45	28.96	49.3%
ParamMerge-Avg	18.91	20.99	19.95	18.38	21.20	19.79	70.06	17.67	43.86	27.87	32.9%
ParamMerge-TA	21.93	22.76	22.34	21.74	23.14	22.44	<u>71.53</u>	21.67	<u>46.60</u>	<u>30.46</u>	<u>71.7%</u>
<i>M-OPD</i>											
Naive M-OPD	20.92	21.60	21.26	17.53	20.99	19.26	68.61	18.67	43.64	28.05	35.6%
Open-MOPD (Ours)	<u>21.98</u>	<u>22.86</u>	<u>22.42</u>	<u>20.84</u>	<u>22.63</u>	<u>21.73</u>	74.49	24.67	49.58	31.24	83.4%

3 Diagnosing the Multi-Teacher Integration Gap

The recipe in Section 2 provides an open baseline for multi-teacher OPD. It combines mixed-domain SFT, one teacher for each domain, hard routing by domain label, and on-policy distillation on the student’s own responses. This setup gives all domains the correct teacher signal while training one shared student. However, the shared student still falls short of the domain-specific teachers. This section studies why the shared student does not retain all of the capabilities learned by the domain teachers. We first establish the integration gap with a simple baseline, then test teacher conflict as a possible cause, and finally measure how much training signal each domain receives and how this signal changes during training.

3.1 A baseline reveals the integration gap

In Table 2, Naive M-OPD lifts π_{mixsft} from 25.67 to 28.05, which shows that the multi-teacher signal is useful. Its score is 0.91 points below π_{mixrl} and 3.50 points below RouteOPD. RouteOPD reaches 31.55, showing that the teachers, the student initialization, and the single-teacher distillation objective are sufficient to learn these capabilities. The score drops when the three teacher signals are naively combined in one student.

Figure 1 shows that the gap is uneven across domains. IF falls 6.16 points below its RouteOPD reference, which is 3.3 times the math gap of 1.89 points. It is also the main source of the total integration gap (Figure 1a). Along the training trajectory (Figure 1b), none of the three domains reaches its corresponding RouteOPD reference within 300 steps. Figure 1c reports the fraction of each domain’s gap to that reference that has been closed. Over the [200, 300]-step window, math closes 64% of its gap, code 40%, and IF only 26%. IF is also the only domain whose score decreases during the middle of training, falling by 11% over the [100, 200] window.

3.2 Testing teacher conflict

A natural hypothesis is that teachers of different domains give mutually contradictory token preferences on the same student trajectory: although the math, code and IF teachers are each selected on their own domain only, they share a large number of formatting words, connectives and reasoning templates. If the teachers disagree on these common tokens, multi-task OPD can push the gradients in different directions and interfere with the shared parameters. To test this hypothesis, multi-teacher on-policy distillation lets all three teachers score the same context for every token sampled by the student. We define

$$c_t = \max_{d \in \mathcal{D}} \log \pi_{\phi_d}(y_t \mid x, y_{<t}) - \min_{d \in \mathcal{D}} \log \pi_{\phi_d}(y_t \mid x, y_{<t}) \quad (5)$$

as the teacher disagreement. If conflicting teacher signals are a major cause of the multi-task OPD failure, c_t should be large for a substantial fraction of tokens.

Figure 2 provides three tests of this hypothesis. The first two examine how often strong disagreement occurs, while the third tests whether changing such tokens improves training. First, disagreement is small and stable. We measured c_t over the full training run. As we can see in Figure 2a, the mean value is 0.126 nat, and c_t remains below 0.27 nat throughout the 300 training steps. Both values are much smaller than the 1-nat conflict threshold (where the probability ratio between the most and least likely teachers is about $e \approx 2.7$). These results do not support widespread teacher conflict as the main cause of the integration gap.

Second, high-conflict tokens are rare. Only 0.62% of tokens satisfy $c_t > 1$ on average. The rate is 3.9% on IF, where disagreement is largest, and 0.31% on math (Figure 2b). A maximum value of 30.58 nat shows that extreme conflict can occur, but it is limited to a small number of tokens.

Third, we tested whether directly changing high-conflict tokens improves training. We used two interventions. A conflict mask removes the top 1%, 5%, or 20% of tokens by c_t from the distillation loss, using thresholds of 0.83, 0.49, and 0.22 nat. We rescaled the remaining weights to keep the total loss scale fixed. A consensus target replaces the hard-routed teacher target with the average of the three teachers’ log-probabilities when $c_t \leq 1$; it keeps the routed teacher when $c_t > 1$ (purple points in Figure 2c).

Under the same settings, the three conflict masks reduce the total score by 0.52, 0.73, and 0.75 points, respectively, relative to the baseline. The consensus target reduces it by 0.83 points. High-conflict tokens may be irrelevant noise, or they may carry domain-specific information. Token-level disagreement alone cannot tell these cases apart. This may explain why removing or replacing such tokens hurts performance.

The teacher-conflict experiments show that token-level teacher disagreement under this metric is unlikely to be the dominant bottleneck. We therefore examine how much training signal each domain provides to the shared student and how the available training resources are distributed across domains.

3.3 Measuring training imbalance across domains

We study how much training signal each domain receives and how this amount changes during training. We first measure the number of valid response tokens and the average per-token reward for each domain. We then examine how these quantities change over the training trajectory. Finally, we study the effect of using old student probabilities in later inner updates from the same rollout batch. These measurements cover three sources of imbalance, namely token counts, reward magnitudes, and updates based on old student probabilities.

Token imbalance across domains. The training loss uses token-mean aggregation, so every valid response token contributes to the loss once. The raw token share of domain d is therefore

$$s_d^{\text{tok}} = \frac{n_d L_d}{\sum_{j \in \mathcal{D}} n_j L_j}, \quad (6)$$

where n_d is the number of prompts of that domain and L_d is the average response length. Figure 3 shows that the two shares are completely decoupled along the whole trajectory, where the prompt share is fixed by

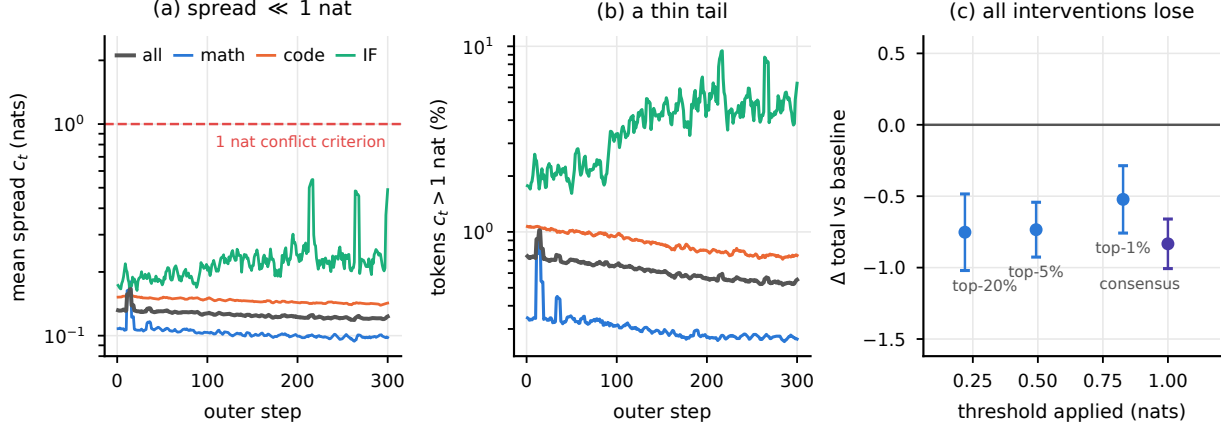


Figure 2 Teacher conflict is measurable but not the bottleneck. (a) Average teacher disagreement c_t (log scale) and its distance to the 1-nat criterion. The legend applies to both panels (a) and (b). (b) Fraction of tokens with $c_t > 1$, broken down by domain. (c) Change in total score for four conflict interventions relative to the baseline. Blue points show the conflict mask, where the horizontal axis indicates the filtering threshold when removing top- $k\%$ tokens by c_t quantile (larger k means lower threshold). Purple points show the consensus method with a fixed threshold of 1 nat; tokens with $c_t \leq 1$ use the average log-probs of all three teachers, while those with $c_t > 1$ keep the routed teacher. Error bars denote validation standard error.

the sampler at 39.8%/39.8%/20.3%, whereas the token share is 49.7%/49.3%/0.99% (Figure 3a). Across all 300 steps the token share of IF never exceeds 1.65% and never falls below 0.44%, so this is not an incidental property of a particular batch. The cause is given directly by length (Figure 3b): the average response length of math and code is about 10,500 tokens while that of IF is only 409 tokens, a difference of more than 25 times; Equation 6 is therefore almost entirely determined by length. This also shows that simply raising the sampling frequency of IF is not a viable fix. For IF to obtain 1/3 of the token budget, its number of prompts would have to be scaled up about 33.6 times (Figure 3c), and the math and code prompts in a batch would be squeezed to the point where long-chain supervision can no longer be maintained.

Reward magnitudes also affect the update budget. In addition to the number of tokens, the strength of each update depends on the reward magnitude. As a simple estimate of the update budget, the contribution of a domain is

$$B_d \propto s_d^{\text{tok}} \bar{m}_d, \quad \bar{m}_d = \mathbb{E}_{t \in d}[|r_t|]. \quad (7)$$

Figure 4a and Equation 7 show how the reward magnitude changes during training. Early in training, $\bar{m}_d$ is 0.019 for math, 0.063 for code, and 0.091 for IF. The largest value is 4.9 times the smallest. Because $\bar{m}_d$ measures the average difference between the student and teacher policies, it should decrease as the student approaches its teacher during training. The decrease is different across domains. Along the same trajectory, IF shrinks by 2.4 times, math by 2.1 times, and code by 1.9 times (Figure 4b). Here $\bar{m}_d$ is the average per-token reward magnitude for domain d . It is the expectation of $|r_t|$ over the tokens in that domain and measures the average effect of one token on the parameter update.

The effect of the reward magnitude becomes clear after the token shares are balanced. When s_d^{tok} is fixed at 1/3 for every domain, the training contribution of each domain depends only on $\bar{m}_d$ in Equation 7. Within 25 steps, the budget share of IF drops from 48.7% to around 9% and ends at 11.4%, while that of code rises from 39.6% to 63.8% (Figure 4c). Thus, balancing the token counts does not keep the training contributions balanced in later steps because the reward magnitudes differ across domains.

Multiple inner updates make the reward stale. To reduce the cost of generation, practical training usually performs K inner updates on one large rollout batch. The teacher remains fixed, while the student changes

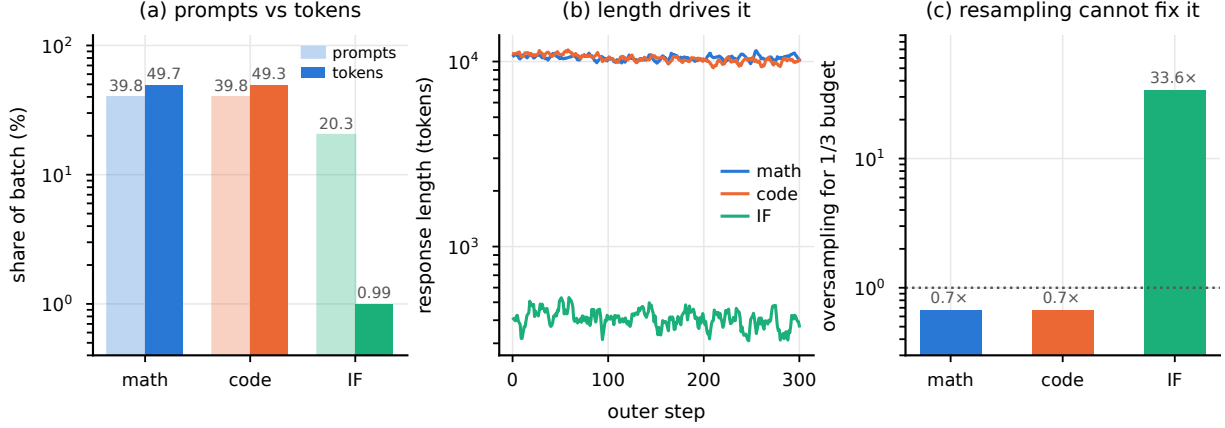


Figure 3 Prompt share and gradient-token share are decoupled. (a) Whole-run averages of prompt share versus token share per domain (log scale; light bars for prompts, solid bars for tokens). IF accounts for $\approx 20\%$ of prompts but only $\approx 1\%$ of gradient tokens. (b) Average response length (log scale) accounts for this difference, with $\approx 10,500$ tokens for math/code versus ≈ 409 tokens for IF. (c) Balancing token share ($1/3$ per domain) solely via oversampling requires a $33.6\times$ prompt multiplier for IF. This severely reduces math and code prompts per batch, making long-chain supervision unsustainable.

after the first inner update. If later updates still use the student probabilities computed during rollout, $\log \pi_\theta(v | x, y_{<t})$ and $\tilde{\pi}_\theta(v | x, y_{<t})$ ($v \in \mathcal{S}_t$), then the student-dependent part of Equation 3 is inconsistent with the current policy. Figure 5 measures the policy shift by the KL between the rollout policy and the current policy within the same batch, and its value rises monotonically with K , growing from 0 at $K=1$ (no shift by definition) to 0.059 at $K=4$ and 0.216 at $K=32$ (Figure 5a); the fraction of tokens clipped by PPO rises with K , from 0 to 0.86 (Figure 5b). Therefore, in a high-throughput setting, most tokens are updated after the student has already drifted away from the rollout policy. The dense reward is still computed from the student and teacher probabilities at rollout time.

These measurements identify three parts of the training signal that need to be controlled. The first is the token budget, followed by reward magnitude and reward freshness. the token budget across domains, the teacher–student gap during training, and the reward delay inside inner updates. The next section presents one method for each part.

4 From Diagnosis to Method: Open-MOPD

Open-MOPD keeps the teacher routing and the on-policy distillation objective of Section 2.1, and changes only how the optimization budget is allocated and how the reward components are computed. The three mechanisms correspond to the three measurements of Section 3.3: token-share balancing controls the domain token budget within a batch, gap-following allocation adjusts the budget over the course of training according to the remaining distillation gap, reward refresh refreshes the student-dependent reward across the several inner updates of one rollout. They act on different time scales, so they can be validated independently and can also be combined into a single training recipe. Figure 6 gives the complete method, showing the data flow of one training iteration, with badges marking the stage that each mechanism rewrites.

4.1 Token-Share Balancing

Token-share balancing assigns a fixed weight to each domain’s token-mean loss. Let g_d^* be the target domain budget, where $\sum_d g_d^* = 1$. In each batch, we compute s_d^{tok} from the attention mask and weight the loss of

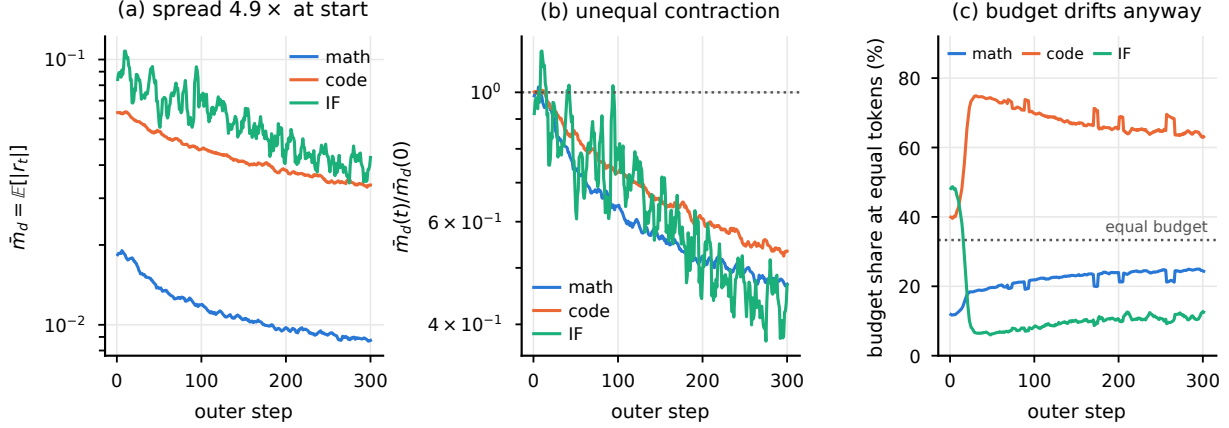


Figure 4 The effective budget drifts with distillation progress. (a) Per-token reward magnitude $\bar{m}_d$ of each domain (log axis), differing by $4.9\times$ early on; (b) after normalizing to their respective initial values, the three domains shrink at different rates, which shows that $\bar{m}_d$ measures the remaining teacher–student gap; (c) on a trajectory where s_d^{tok} is flattened to $1/3$, Equation 7 degenerates to $B_d \propto \bar{m}_d$, so the curves show the budget drift caused by the gap alone. Within 25 steps the share of IF falls below the dashed line to about 9%, while code rises to 63.8%. Flattening the token share does not lock the budget in place.

domain d by

$$w_d^{\text{share}} = \frac{g_d^*}{s_d^{\text{tok}}}. \quad (8)$$

After this weighting, the effective share of domain d is

$$\frac{w_d^{\text{share}} s_d^{\text{tok}}}{\sum_j w_j^{\text{share}} s_j^{\text{tok}}} = g_d^*.$$

This directly controls the actual token ratio in the loss, without relying on stable response lengths or oversampling short answers.

The main recipe uses the equal-share target $g^* = (1/3, 1/3, 1/3)$, which needs no corpus-specific tuning. On the token shares shown in Figure 3, it yields weights of 0.69 for math, 0.66 for code and 32.7 for IF. Every IF token is amplified about 48 times to compensate for its 25-fold length disadvantage, while Equation 8 guarantees that the weighted shares of the three domains are exactly 33.33%. Compared with changing only the prompt sampling rates, token-share balancing preserves the diversity of math and code prompts while giving the shorter IF responses a meaningful contribution to training.

Token-share balancing changes how much each domain contributes to the loss through its token weight; it does not change the sampling frequency or the reward magnitude of individual tokens. We intentionally keep this design because the size of the reward shows the gap between the teacher and the student, and simply normalizing it would remove useful information about learning progress.

4.2 Gap-Following Allocation

Let m_d be the running per-token reward magnitude for domain d (for example, an exponential moving average of $|r|$), which serves as an observable proxy for the remaining teacher–student gap. A naive idea is to normalize the loss with $m_d^{-\alpha}$, giving larger weights to domains with smaller rewards. This may seem reasonable early in training because a small reward can indicate that a domain is learning slowly. However, m_d also tracks the remaining distillation gap: as a domain approaches its teacher, its m_d becomes smaller. Inverse normalization therefore assigns more budget to domains that have already made more progress.

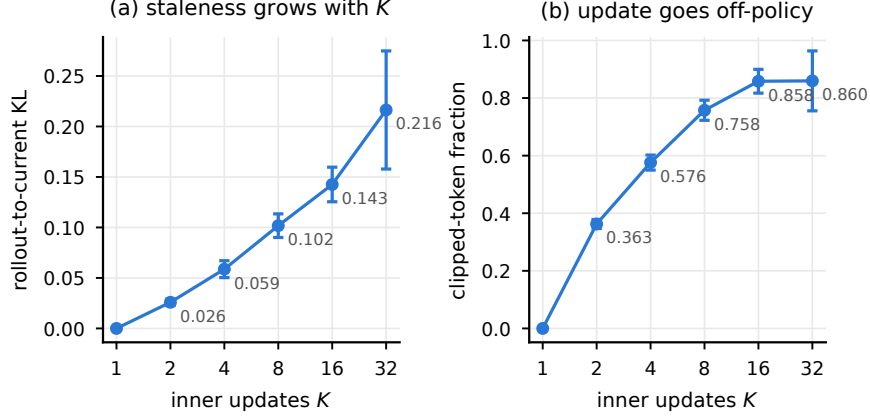


Figure 5 Repeated inner updates change the student policy within a rollout batch. The horizontal axis is the number of inner updates K per rollout batch. (a) Within the same batch, the KL between the rollout policy and the current student policy grows monotonically with K ; (b) the fraction of tokens clipped by PPO rises with K . At $K=1$ both are 0, and the dense reward is then consistent with the current student.

This forms an unstable feedback loop. As a domain converges, its m_d decreases, which in turn increases its assigned weight $m_d^{-\alpha}$. The domain then receives even more training budget, accelerating its convergence and shrinking m_d further. Without any balancing force, the weights continuously diverge and eventually crash the training. We observe this pattern directly in experiments: over the first 75 steps, m_d for IF decreases 35.3-fold. Setting $\alpha = 0.5$ causes the IF weight to rise from 26.7 to 90.7, while the code weight drops from 0.44 to 0.27. Therefore, we believe that the inverse rule is unstable when α is positive and not too small.

Gap-following allocation keeps the direction of the gap, interprets it as “capability not yet distilled”, and allocates the budget to the domains that still have a larger gap.

$$\tilde{w}_d = w_d^{\text{share}} \cdot \text{Clamp}\left(\left(\frac{m_d}{m_{\text{ref}}}\right)^\alpha, 0.05, 20\right), \quad w_d^{\text{gap}} = \frac{\tilde{w}_d}{\sum_j \tilde{w}_j s_j^{\text{tok}}}, \quad (9)$$

Here $\tilde{w}_d$ is the unnormalized weight before the final normalization. It combines the token-share weight w_d^{share} from the previous section with a clipped gap factor. The reference value m_{ref} is the mean of m_d across domains in the current batch. Thus, $(m_d/m_{\text{ref}})^\alpha$ measures the reward magnitude of domain d relative to the other domains. We clip this factor to $[0.05, 20]$ so that a sudden change in one domain’s reward does not make its training weight too small or too large. The divisor $\sum_j \tilde{w}_j s_j^{\text{tok}}$ is the token-share-weighted mean of the weights, so after normalization $\sum_d w_d^{\text{gap}} s_d^{\text{tok}} = 1$ and the total loss scale of a batch stays unchanged. Consequently, when a domain approaches its teacher, its m_d and its budget fall together, while domains with a larger teacher–student gap receive more updates. This allocation rule therefore follows the remaining gap rather than normalizing reward magnitudes; Section 5.2 shows its benefit on top of token-share balancing and the collapse that occurs when α takes a negative sign.

4.3 Reward Refresh

The first two mechanisms deal with the budget between domains; reward refresh deals with temporal inconsistency inside one rollout. Our design draws on an insight from AsyncOPD [6]: when the student changes, the student-dependent part of a reverse-KL signal should be recomputed with the current student. We apply this insight to a different source of staleness by refreshing the student-dependent reward before each of the K inner updates in our synchronous multi-teacher pipeline. The student is updated K times within one rollout batch, while the rollout trajectories come from the student before these updates. This creates a mismatch in the reward, which reward refresh corrects. Suppose that one rollout is used for K

Overview of Open-MOPD

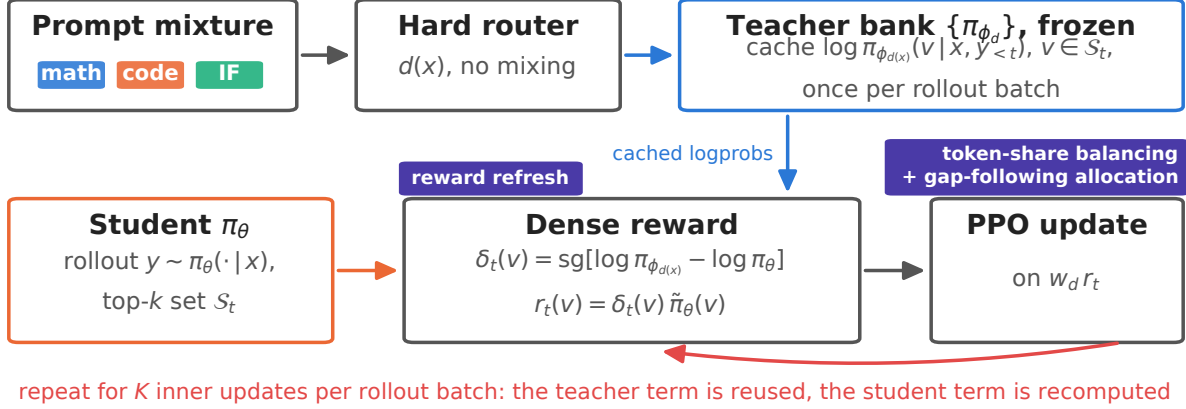


Figure 6 Overview of Open-MOPD. Prompts are hard-routed by domain label. During rollout, the teacher computes and caches log-probabilities on S_t once. The student generates responses under its own distribution. Together, both terms form the dense reward for PPO updates. Within each rollout batch, the inner update repeats K times, reusing the teacher term while recomputing the student term via reward refresh. Badges highlight where each mechanism acts: token-share balancing and gap-following allocation determine the domain loss weight w_d , while reward refresh rebuilds r_t in every inner update.

update minibatches. The teacher needs only one prefill to compute $\log \pi_{\phi_d(x)}(v | x, y_{<t})$, while the student probabilities are recomputed after each update. In the actor forward of the k -th inner update, reward refresh recomputes the student-dependent term on the same set S_t .

$$\begin{aligned} \delta_t^{(k)}(v) &= \text{sg}\left[\log \pi_{\phi_d(x)}(v | x, y_{<t}) - \log \pi_\theta^{(k)}(v | x, y_{<t})\right], \\ r_t^{(k)}(v) &= \delta_t^{(k)}(v) \cdot \tilde{\pi}_\theta^{(k)}(v | x, y_{<t}), \quad v \in S_t, \quad k = 0, \dots, K-1. \end{aligned} \quad (10)$$

Here $\tilde{\pi}_\theta^{(k)}$ is computed on the current student log-probabilities following Equation 3. It is then aggregated into $r_t^{(k)}$ by Equation 4 and placed in the advantage slot of PPO. The update is subsequently constructed with the $w_d r_t^{(k)}$ of the current domain, where $w_d = w_d^{\text{gap}}$ when gap-following allocation is enabled and $w_d = w_d^{\text{share}}$ otherwise. Reward refresh reuses the student forward pass already required by PPO and adds no teacher forward. It therefore makes each update use the difference between the current student and the teacher.

At $K = 1$, Eqs. (3), (4), and (10) reduce to the objective in Section 2.1; at $K > 1$, they repair only the part of the reward that explicitly depends on the student. The sampled student states still come from rollout time, so reward refresh removes the staleness that can be removed without another prefill.

4.4 The Complete Algorithm

Table 3 summarizes the three mechanisms and the quantities they modify. Together, they change token weighting, domain weighting, and reward evaluation in the shared student.

Algorithm 1 summarizes the complete training loop. The teacher log-probabilities are computed once per rollout, while the student-dependent reward term is recomputed before each inner update.

Together, these three mechanisms address the problems found in our diagnosis. Token-share balancing fixes the token budget, gap-following allocation changes each domain’s share of updates as training progresses, and reward refresh keeps the feedback up to date.

Table 3 Summary of the three Open-MOPD mechanisms. Each mechanism modifies a different part of the training computation.

Mechanism	Measurement	Intervention
token-share balancing	Response-token share s_d^{tok} in the current batch	Set $w_d^{\text{share}} = g_d^*/s_d^{\text{tok}}$ to equalise the domains’ token shares in the loss.
gap-following allocation	Running mean reward magnitude m_d for each domain	Multiply the share weight by the clipped factor $(m_d/m_{\text{ref}})^\alpha$, giving more weight to domains with a larger current reward signal.
reward refresh	Student log-probabilities at each inner update	Recompute the student-dependent reward while reusing the cached teacher log-probabilities.

Algorithm 1 Open-MOPD training loop

Require: Domain-mixed sampler, student π_θ , teachers $\{\pi_{\phi_d}\}$, inner-step count K

Ensure: Updated student π_θ

```

1: for each training step  $t$  do
2:   Sample rollout batch  $\mathcal{R}_t$  from  $\pi_\theta$  and route prompts by domain label.
3:   Compute teacher log-probabilities  $\ell_{\phi, d(x^{(b)})}^{(b)}$  for each sampled response.
4:   Compute response-token shares  $s_d^{\text{tok}}$  and weights  $w_d^{\text{share}}$  using Equation 8.
5:   Update reward means  $m_d$  and compute gap weights  $w_d$  using Equation 9.
6:   for each inner minibatch  $\mathcal{M}_{t,k}$ ,  $k = 0, \dots, K - 1$  do
7:     Recompute current-student log-probabilities and refresh  $r_t^{(k)}$  using Equation 10.
8:     Update  $\theta$  with the PPO objective using  $w_d r_t^{(k)}$ .
9:   end for
10: end for

```

5 Ablation Studies

Token-share balancing controls the response-token share across domains, gap-following allocation controls how the update budget follows the remaining teacher–student gap, and reward refresh recomputes the student-dependent reward before each inner update. We first verify these effects one mechanism at a time and measure the resulting change in the corresponding domain score. We then evaluate the combined recipe by its reduction of the integration gap. For reward refresh, we also measure the runtime cost.

5.1 Token-Share Balancing

The first test checks the mechanism directly. With token-share balancing enabled (Section 4.1), the weighted token share $w_d s_d^{\text{tok}}$ stays at 33.33% over all 300 steps (marked by the dashed line in Figure 7a); the same sampler with it switched off hands 99% of the gradient tokens to math and code and leaves only about 1% to IF (Figure 3a). The first cause of imbalance measured in Section 3.3 is therefore removed entirely. In Table 4, token-share balancing gives a gain of +1.17 points, almost entirely from IF; math and code barely move. This shows that, under naive M-OPD, most token loss is spent on math and code, where the student is already close to its teacher. Token-share balancing increases the token share of IF, which receives very little training otherwise, and improves the overall M-OPD score.

5.2 Gap-Following Allocation

We add gap-following allocation on top of token-share balancing. Unlike static balancing, which keeps every domain at around 1/3, gap-following allocation changes domain ratios dynamically as training goes on (Figure 7a). In this setting, the budget goes to the domain with the largest teacher–student gap $\bar{m}_d$. Averages calculated over the middle window of the baseline trajectory show that $\bar{m}_d$ is 0.028 for code, 0.008 for math, and 0.003 for IF ($2.16\times$, $0.61\times$, and $0.23\times$ the average, respectively). From the first 25 steps to steps 200–300,

Table 4 From Naive M-OPD to the full recipe. Each row changes one thing from the row above, as shown in the first column. The three middle columns show whether token-share balancing, gap-following allocation, and reward refresh are turned on. K is the number of inner updates per rollout batch.

Configuration	Mechanism			Six datasets				
	share	gap	refresh	K	Math	Code	IF	Total Δ
Naive M-OPD				1	21.26	19.26	43.64	28.05 —
+share	✓			1	20.55	19.57	47.53	29.22 +1.17
+gap	✓	✓		1	21.00	19.31	49.50	29.94 +1.89
<i>Switching to the $K=4$ throughput setting (256 prompts per update, $4\times$ rollout batch)</i>								
Naive M-OPD (same-setting control)				4	21.62	19.72	46.49	29.28 +1.23
+share+gap	✓	✓		4	23.05	21.07	47.16	30.43 +2.38
Open-MOPD	✓	✓	✓	4	22.42	21.73	49.58	31.24 +3.19

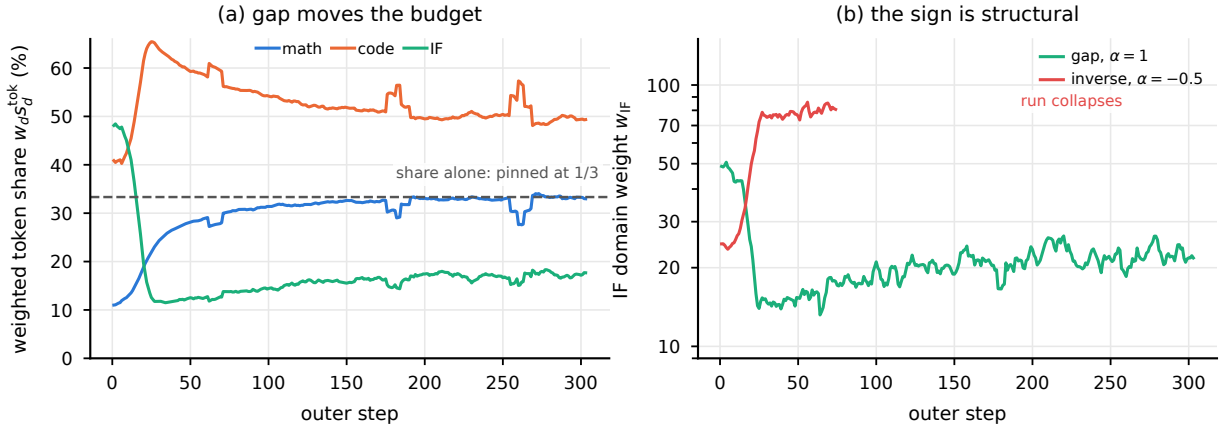


Figure 7 Gap-following allocation dynamically changes the budget trajectory. (a) Weighted token share $w_d s_d^{\text{tok}}$: static token-share balancing keeps the share fixed at 1/3 (dashed line). Adding gap-following allocation ($\alpha=1$) dynamically adjusts the budget based on the remaining teacher-student gap. Domains that converge quickly lose budget, while those with larger gaps receive more resources (e.g., code exceeds 50%, whereas IF drops to $\sim 17\%$). (b) IF domain weight (log scale): setting $\alpha=1$ properly reduces its weight, whereas the inverse formulation ($\alpha = -0.5$) creates an unstable feedback loop, causing training to collapse at step 74.

the math share grows from 15.1% to 32.6%, the IF share drops from 34.5% to 17.1%, and code gets the most budget (averaging 55.4% and peaking at 87.6%). When tested alone without reward refresh, this mechanism brings a +0.72-point improvement ($K = 1$ ladder, Table 4).

Reversing the factor gives more budget to domains with smaller gaps. This favors domains that the student already handles well and creates a positive feedback loop. In our experiment, the IF gap shrinks by $32.5\times$, its weight rises from 24.4 to 80.9, and training stops at step 74 (Figure 7b). The sign of α is therefore fixed by the direction of the gap.

5.3 Reward Refresh

Reward refresh (Section 4.3) moves the student-dependent part of the dense reward into every inner update. The teacher log-probs are still prefilled once at the top of the rollout batch and reused throughout, and the student log-probabilities are read from the actor forward that PPO already performs. Reward refresh therefore adds no extra student forward. Without refresh, the dense reward uses the student probabilities saved at rollout time; with refresh, it uses the student probabilities from the current inner update. The teacher term is computed once per outer step in both cases; only the student parameters used to evaluate the reward change.

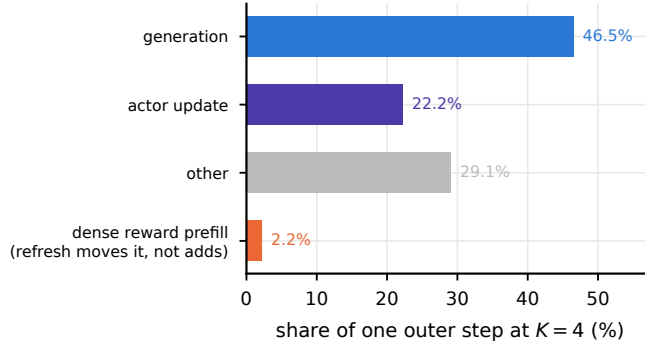


Figure 8 Composition of one outer step at $K=4$. Averaged over steady training steps. The dense-reward computation accounts for 2.2% of each step. Reward refresh changes only when the student probabilities are read for this computation, without changing total throughput.

We measure the extra runtime and performance gain from reward refresh. At $K=4$, the dense-reward computation takes 27.3s with refresh and 27.8s without refresh. These values account for 2.10% and 2.12% of one outer step, respectively (Figure 8). The full step takes 1298s with refresh and 1313s without it. The difference is small and falls within the step-to-step variation, so **reward refresh adds no measurable extra runtime cost**. On top of token-share balancing and gap-following allocation, reward refresh gives a further +0.81 points at $K=4$ (Table 4). This gain supports the effectiveness of the mechanism and completes the Open-MOPD recipe.

6 Related Work

6.1 On-Policy Distillation and Multi-Teacher Extensions

Distillation on a fixed corpus. Knowledge distillation trains a student to match the teacher’s output distribution [8]. For sequence models, the teacher usually generates responses in advance, and the student learns from this fixed corpus [9]. The student therefore trains on prefixes produced by the teacher, while inference uses prefixes produced by the student. This distribution shift creates the exposure-bias problem in sequence prediction [10, 11]. A large difference in teacher and student capacity can also make distillation difficult [12–14].

On-policy distillation. On-policy distillation (OPD) trains the student on its own rollouts and uses teacher feedback on the prefixes that the student actually visits [15, 16]. Recent work improves this process by changing the divergence, the sampling rule, or the token-level training signal [3, 17–19]. Other studies examine training stability and failure modes, including unreliable feedback on long or drifted prefixes [20]. A recent survey organizes these methods by their feedback signal, teacher access, and optimization rule [21]. Most existing work still uses one teacher. Our work studies how several teachers share the updates of one student.

Staleness under batch reuse. PPO supports multiple minibatch updates within one rollout cycle [22]. In asynchronous RL, policy lag occurs when rollouts come from a policy that is several updates behind the learner [23–28]. Prior work studies how much stale data RL systems can tolerate [29, 30], and recent work examines staleness in OPD [6, 31]. In our synchronous pipeline, each rollout batch is partitioned into K minibatches and used for sequential student updates, so the student-dependent reward can become stale as the student changes. Reward refresh recomputes this term before each inner update using the student probabilities from PPO’s existing actor forward, adding no extra student forward.

From Single-Teacher to Multi-Teacher OPD. MOPD extends OPD to multiple domain teachers through a standard three-stage recipe: train domain specialists from a shared SFT model, route each student rollout to

its domain teacher, and use token-level teacher feedback for training [4, 32]. This recipe has been used in several public models. Nemotron-Cascade 2 distills strong intermediate teachers to recover capabilities lost during Cascade RL, while Agents-A1 combines six domain teachers and normalizes the loss across responses and domains [33, 34]. At a larger scale, DeepSeek-V4 distills more than ten teachers, and Kimi K3 uses nine teachers defined by domain and reasoning effort [35, 36]. These studies show that MOPD can integrate specialists at different model scales. Open-MOPD studies how to balance the training received by different domains during this integration.

6.2 Integrating Domain Experts

Training Domain Specialists. Reinforcement learning on a single domain can produce a strong specialist for that domain. The success of DeepSeekMath and DeepSeek-R1 in mathematical reasoning provided a practical recipe for LLM reinforcement learning [1, 37]. DAPO further developed this recipe for large-scale training [2]. Domain-specific RL has since expanded to software engineering, search, and instruction following [38–41]. Each pipeline produces one specialist, while deployment usually requires one model that can handle all of these domains. Our work studies how to integrate these specialists into one student through multi-teacher OPD.

Integration in data space. A common way to combine domains is to train one model on mixed-domain data. Qwen3, for example, uses general-domain RL to improve a wide range of tasks [42]. Another approach trains the domains in sequence. Nemotron-Cascade applies a separate RL stage to each domain, so each stage can use its own data and training settings [43]. In joint training, the data mixture can strongly affect the final model. DoReMi studies this problem in pre-training, while MoDoMoDo extends data-mixture optimization to multi-domain RLVR [44, 45]. Equal task sampling still does not guarantee equal training: different tasks can produce gradients with very different magnitudes [46]. Open-MOPD studies how unequal response-token counts, different teacher–student gaps, and outdated rewards create imbalance across domains in multi-teacher OPD.

Integration in weight space and in module space. A second route combines domain experts after they have been trained. Weight-space methods merge task-specific parameter changes into one checkpoint. Task Arithmetic introduced this operation, TIES-Merging resolves conflicting parameter signs, and DARE sparsifies model changes before they are merged [47–49]. Other methods retain the expert structure during integration. BTM trains experts on different domains and combines them through ensembling or parameter averaging, BTX turns their feed-forward layers into a routed mixture of experts, and BTS connects frozen experts with lightweight stitch layers [50–52]. These methods integrate specialists after separate training. Multi-teacher OPD trains one shared student from their outputs, and Open-MOPD studies how to balance the updates received by different domains during this shared training.

7 Conclusion

We build from scratch a fully open multi-teacher on-policy distillation pipeline—mixed-domain SFT, three domain RL teachers, multi-teacher OPD—and use it to answer one concrete question, namely when routing is already error-free, what prevents the capabilities of three experts from being written into the same set of parameters at once. Our experiments identify the allocation of the optimization budget as the main bottleneck. We identify three separable contributors to imbalance: unequal token counts, unequal reward magnitudes, and outdated rewards during repeated inner updates. The three mechanisms of Open-MOPD—**token-share balancing**, **gap-following allocation**, and **reward refresh**—correspond one by one to these three sources and are validated separately in ablation experiments. Together, they reduce the integration gap from 3.50 points to 0.31 points, while the recovery rate relative to RouteRL rises from **35.6%** to **83.4%**. We release the complete recipe and the mechanism implementations to support reproducible follow-up work.

References

- [1] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024.
- [2] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale, 2025.
- [3] Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, et al. Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe, 2026.
- [4] Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, et al. MOPD: Multi-Teacher On-Policy Distillation for Capability Integration in LLM Post-Training, 2026.
- [5] Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, et al. Llama-Nemotron: Efficient Reasoning Models, 2025.
- [6] Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjun Kang, Sanghyun Park, Donghoon Kim, Minjae Lee, Minseo Kim, et al. AsyncOPD: How Stale Can On-Policy Distillation Be?, 2026.
- [7] Hugging Face. SmolLM3: Smol, multilingual, long-context reasoner. <https://huggingface.co/blog/smollm3>, 2025.
- [8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network, 2015.
- [9] Yoon Kim and Alexander M. Rush. Sequence-Level Knowledge Distillation, 2016.
- [10] Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence Level Training with Recurrent Neural Networks, 2015.
- [11] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks, 2015.
- [12] Jang Hyun Cho and Bharath Hariharan. On the Efficacy of Knowledge Distillation, 2019.
- [13] Seyed-Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, and Hassan Ghasemzadeh. Improved Knowledge Distillation via Teacher Assistant, 2019.
- [14] Yuetai Li, Xiang Yue, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, Bhaskar Ramasubramanian, and Radha Poovendran. Small Models Struggle to Learn from Strong Reasoners, 2025.
- [15] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: On-Policy Distillation of Large Language Models, 2023.
- [16] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes, 2023.
- [17] Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. DistiLLM: Towards Streamlined Distillation for Large Language Models, 2024.
- [18] Jongwoo Ko, Tianyi Chen, Sungnyun Kim, Tianyu Ding, Luming Liang, Ilya Zharkov, and Se-Young Yun. DistiLLM-2: A Contrastive Approach Boosts the Distillation of LLMs, 2025.
- [19] Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative Knowledge Distillation: Bridging the Teacher-Student Gap Through Interleaved Sampling, 2024.
- [20] Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Jiakai Liu, Zhuo Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting On-Policy Distillation: Empirical Failure Modes and Simple Fixes, 2026.
- [21] Mingyang Song and Mao Zheng. A Survey of On-Policy Distillation for Large Language Models, 2026.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, 2017.
- [23] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, et al. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning, 2025.

- [24] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, et al. StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation, 2025.
- [25] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, et al. Laminar: A Scalable Asynchronous RL Post-Training Framework, 2025.
- [26] Ran Yan, Youhe Jiang, Tianyuan Wu, Jiaxuan Gao, Zhiyu Mei, Wei Fu, Haohui Mai, Wei Wang, Yi Wu, and Binhang Yuan. AReaL-Hex: Accommodating Asynchronous RL Training over Heterogeneous GPUs, 2025.
- [27] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, et al. RollPacker: Mitigating Long-Tail Rollouts for Fast, Synchronous RL Post-Training, 2025.
- [28] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A Flexible and Efficient RLHF Framework, 2024.
- [29] Haizhong Zheng, Jiawei Zhao, and Beidi Chen. Prosperity before Collapse: How Far Can Off-Policy RL Reach with Stale Data on LLMs?, 2025.
- [30] Xiaocan Li, Shiliang Wu, and Zheng Shen. A-3PO: Accelerating Asynchronous LLM Training with Staleness-aware Proximal Policy Approximation, 2025.
- [31] Liwen Zheng and Haiyun Jiang. Blockwise Policy-Drift Gating for On-Policy Distillation, 2026.
- [32] Core Team, Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, et al. MiMo-V2-Flash Technical Report, 2026.
- [33] Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, Renjie Pi, Grace Lam, Nayeon Lee, Alexander Bukharin, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nemotron-Cascade 2: Post-Training LLMs with Cascade RL and Multi-Domain On-Policy Distillation, 2026.
- [34] Lei Bai et al. Scaling the Horizon, Not the Parameters: Reaching Trillion-Parameter Performance with a 35B Agent, 2026.
- [35] DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, et al. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, 2026.
- [36] Kimi Team. Kimi K3: Open Frontier Intelligence, 2026.
- [37] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025.
- [38] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents, 2025.
- [39] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution, 2025.
- [40] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning, 2025.
- [41] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing Verifiable Instruction Following, 2025.
- [42] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. Qwen3 Technical Report, 2025.
- [43] Boxin Wang, Chankyu Lee, Nayeon Lee, Sheng-Chieh Lin, Wenliang Dai, Yang Chen, Yangyi Chen, Zhuolin Yang, et al. Nemotron-Cascade: Scaling Cascaded Reinforcement Learning for General-Purpose Reasoning Models, 2025.
- [44] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining, 2023.
- [45] Yiqing Liang, Jielin Qiu, Wenhao Ding, Zuxin Liu, James Tompkin, Mengdi Xu, Mengzhou Xia, Zhengzhong Tu, Laixi Shi, and Jiacheng Zhu. MoDoMoDo: Multi-Domain Data Mixtures for Multimodal LLM Reinforcement Learning, 2025.

- [46] Runzhe Wu, Ankur Samanta, Ayush Jain, Scott Fujimoto, Jeongyeol Kwon, Ben Kretzu, Youliang Yu, Kaveh Hassani, Boris Vidolov, and Yonathan Efroni. Imbalanced Gradients in RL Post-Training of Multi-Task LLMs, 2025.
- [47] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing Models with Task Arithmetic, 2022.
- [48] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving Interference When Merging Models, 2023.
- [49] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch, 2023.
- [50] Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tim Althoff, Noah A. Smith, and Luke Zettlemoyer. Branch-Train-Merge: Embarrassingly Parallel Training of Expert Language Models, 2022.
- [51] Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, et al. Branch-Train-MiX: Mixing Expert LLMs into a Mixture-of-Experts LLM, 2024.
- [52] Qizhen Zhang, Prajjwal Bhargava, Chloe Bi, Chris X. Cai, Jakob Foerster, Jeremy Fu, Punit Singh Koura, Ruan Silva, Sheng Shen, Emily Dinan, Suchin Gururangan, and Mike Lewis. BTS: Harmonizing Specialized Experts into a Generalist LLM, 2025.

Appendix

A Constructing the Open-MOPD Recipe

In this section, we introduce how we obtained the final Open-MOPD recipe. We first screen on math SFT for a base model that stably produces reasoning trajectories, and then extend the selected base model to a mixed-domain SFT over math, code and instruction following.

A.1 Base Model Choice

OpenR1 math comparison between 1.7B and 7B. Early screening applied the same 93,733 OpenR1 math examples to Qwen3-1.7B-Base and Qwen2.5-7B-Base separately. Both runs use a 32,768-token SFT length limit, a global batch of 128 and a learning rate of 4×10^{-5} , with 732 steps per epoch. Table 6 reports the complete curve over the first five epochs.

The problem with Qwen3-1.7B-Base is not a lack of generation length: under the 31K budget, 69.17–80.42% of the samples still exhaust the limit at all five checkpoints, and only 20.00–31.25% of the answers close `</think>`; adding epochs does not produce a monotone improvement either, and AIME24 stays below 7.1%. This means that a large fraction of the trajectories seen by a subsequent RL or OPD stage contain only a reasoning prefix and never reach a final answer. A drop in task score can then not be attributed uniquely to the reward, the routing or cross-domain interference, because the base model and the data mixture themselves have not yet learnt to end a reasoning trajectory reliably.

The same OpenR1 data shows the opposite trend on Qwen2.5-7B-Base: AIME24 rises from 16.25% to 31.25% and the truncation rate falls from 42.92% to 12.92%. This rules out the explanation that “OpenR1 trajectories are inherently too long, so any base model would truncate”, and it shows that enlarging the capacity of the base model does recover trajectory closure. Qwen2.5-7B-Base therefore satisfies the capability condition, but it puts the student, the three RL teachers and every mechanism ablation of the full recipe at 7B scale; for an open research baseline that has to be run repeatedly, this cost is too high.

Choosing SmolLM3-3B-Base. The final recipe selects SmolLM3-3B-Base as the compromise between the two ends. Four epochs of mixed-domain SFT over the three domains give π_{mixsft} : OpenR1-Math-93k, the OCR-50k subset sampled from the full OpenCodeReasoning set, and Instruction-Nemotron contribute roughly 37.3%, 28.1% and 34.6% of the training response tokens respectively. The per-domain sample counts are balanced by response token count, so that the 820K short IF answers do not drown out the fewer but longer math and code trajectories.

Table 5 summarises the selection evidence for the three candidate base models. From the table, we can see that 3B is therefore not an arbitrary choice of a “medium model” but an operating point bracketed by two failure boundaries: going down to 1.7B, generation is dominated by unclosed reasoning and truncation, and mechanism attribution is no longer unique; going up to 7B, the single model is stronger, but the end-to-end multi-teacher recipe and its repeated ablations exceed the resource boundary we set for a community baseline. SmolLM3-3B-Base retains enough trajectory capacity while still allowing the final Open-MOPD training to run on a single 8×A100-80GB node.

A.2 Stage Hyperparameters

This section lists all hyperparameters of the four stages of the recipe, taken from the training configurations that were actually run rather than from recommended values compiled after the fact. Table 7 gives the mixed-domain SFT and the three domain RL teachers, and Table 8 gives single-domain OPD (the RouteOPD oracle of the main text) and multi-teacher OPD. Items not listed use the verl defaults.

Mixed-domain SFT and domain RL teachers. The three teachers share one GRPO configuration and differ only in data, sequence length, rollout group size and number of steps: IF answers are short, so it uses a smaller response limit, smaller groups and more steps; math and code use a 30,000-token response limit to

Table 5 Base-model choice for Open-MOPD. The table summarises directly the experimental results used when constructing the recipe. The truncation evidence is reported in a dedicated row below each base model.

Base model	SFT data	Performance
Qwen3-1.7B-Base	OpenR1-Math-93k	AIME24 best 7.08
<i>Truncation evidence:</i> 69.17–80.42%; capacity and trajectory closure insufficient.		
SmolLM3-3B-Base	Three-domain mixSFT	AIME24 15.63
<i>Truncation evidence:</i> 15.2%; chosen as the base model of the recipe.		
Qwen2.5-7B-Base	OpenR1-Math-93k	AIME24 31.25
<i>Truncation evidence:</i> 12.92%; capable enough, but a full multi-teacher ablation is too costly.		

Table 6 Base-model screening under the same OpenR1-Math-93k SFT. Each cell is AIME24 accuracy / generation truncation rate (%). Qwen3-1.7B-Base mainly generates long, unclosed reasoning trajectories at every observed checkpoint; Qwen2.5-7B-Base instead raises accuracy and lowers truncation together as SFT proceeds.

Epoch	Step	Qwen3-1.7B-Base	Qwen2.5-7B-Base
1	732	4.58 / 75.42	16.25 / 42.92
2	1464	7.08 / 72.50	20.83 / 35.42
3	2196	6.67 / 80.42	30.83 / 22.08
4	2928	5.83 / 69.17	30.42 / 19.58
5	3660	6.67 / 69.58	31.25 / 12.92

accommodate the full reasoning chain. All three disable the KL penalty and apply group filtering on accuracy (the samples of one prompt are discarded if they are all correct or all wrong, with at most 8 generation batches resampled, which the dynamic sampling trick proposed by DAPO), so that the gradient comes only from groups that discriminate.

Single-domain OPD and multi-teacher OPD. The two share the same optimiser and distillation settings, so the difference between the RouteOPD oracle and M-OPD in the main text comes only from the number of teachers and the domain mixture, not from the training configuration. The dense reward is computed on the student top- k ($k=16$), and $K=\text{train batch}/\text{mini batch}=4$ is the off-policy depth. The multi-teacher column additionally enables the domain sampler and the three mechanism switches: token-share balancing sets the target gradient share, gap-following allocation adjusts the domain weights by the remaining gap, and reward refresh refreshes the dense reward within every inner update; the table gives their values in the final recipe.

Table 7 Hyperparameters of stages one and two. SFT is standard supervised fine-tuning; the three RL teachers are all GRPO, each initialised from π_{mixsft} and trained only on the verifiable reward of its own domain. World size is given as number of nodes $\times$ GPUs per node, all A100-80GB. “—” means the item does not apply to that stage.

	Mixed-domain SFT	RL: math	RL: code	RL: IF
World size	8 $\times$ 8	8 $\times$ 8	8 $\times$ 8	4 $\times$ 8
Starting point	SmolLM3-3B-Base		π_{mixsft}	
Algorithm	Cross-entropy		GRPO	
train batch size	128	128	128	128
mini batch size	—	32	32	32
Learning rate	4×10^{-5}		1×10^{-6}	
lr schedule	cosine, warmup 3%	constant, warmup 10 steps		
grad clip	0.2		1.0	
Prompt limit	—	1,024	2,048	2,048
Response limit	32,768 (total length)	30,000	30,000	2,048
rollout n	—	16	16	8
Temperature	—		1.0	
epochs / steps	4 epochs	1,000 steps	1,000 steps	3,000 steps
clip _{low} /clip _{high}	—		0.2 / 0.25	
KL penalty	—		disabled	
entropy coeff	—		0	
group filtering	—	by acc, at most 8 generation batches		

Table 8 Hyperparameters of stage three. The left columns are the three single-domain OPD runs (the RouteOPD oracle of the main text) and the right column is multi-teacher OPD. The two share the optimiser, sequence lengths and distillation settings; the token-share balancing / gap-following allocation / reward refresh rows are the values of our mechanisms in the final recipe.

	Single-domain OPD (RouteOPD oracle)			Multi-teacher OPD
	math	code	IF	Open-MOPD
World size	1 $\times$ 8	1 $\times$ 8	1 $\times$ 8	1 $\times$ 8
Teacher	$\pi_{\phi_{\text{math}}}$	$\pi_{\phi_{\text{code}}}$	$\pi_{\phi_{\text{IF}}}$	3-teacher hard routing
Student starting point				π_{mixsft}
train batch size				1,024
mini batch size				256 (i.e. $K=4$)
Learning rate				1.5×10^{-6} , constant
clip _{low} /clip _{high}				0.2 / 0.28
KL penalty				disabled
Advantage estimation				dense reward written directly into the advantage slot
Prompt limit	1,024	2,048	2,048	2,048
Response limit	16,384	16,384	1,024	16K (math/code) / 2K (IF)
Total steps				600
Reward form		single-teacher dense reward		multi-teacher hard-routed dense reward
top- k		16, from the student distribution, nucleus truncation $p=0.99$		
Reward weighting		weighted by student probability $\tilde{\pi}_{\theta}$ (Eq. 3)		
share target g^*	—	—	—	(1/3, 1/3, 1/3)
gap-following α	—	—	—	1.0 (m_d/m_{ref} forward)
reward refresh		enabled		enabled